

INSTITUTO TECNOLÓGICO DE COSTA RICA

Campus Tecnológico Central Cartago
IC-3101 Arquitectura de Computadores
Estudiante: Allan Bolaños Barrientos
Número de carné 2024145458
Profesor: Ms.c Esteban Arias Mendez

Ingeniería en Computación
II Semestre 2024
Estudiante 2: Brian Ramírez Arias
Número de carné: 2024109557

Proyecto 1 — Tricount en NASM

Fecha de entrega: 10/23/24

1 Descripción del programa

El proyecto consiste en desarrollar una aplicación de administración de gastos llamada "Tricount Terminal", utilizando el lenguaje de programación NASM en una terminal de Linux. La aplicación permitirá gestionar gastos compartidos entre un grupo de amigos, facilitando la entrada de datos sobre amigos y gastos, así como la conciliación de cuentas. Incluirá funcionalidades como la validación de errores, la persistencia de datos mediante almacenamiento en archivos, y la generación de reportes. Además, se garantizará una interfaz textual clara y amigable, con mensajes informativos para el usuario, todo en un entorno de programación que fomenta el trabajo colaborativo y el uso de Git para el control de versiones.

2 Interfaz de usuario

2.1 Comando para inicializar

Es necesario instalar la librería de "xdotool". - `sudo apt install xdotool`

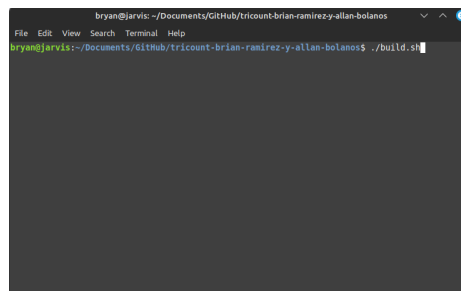
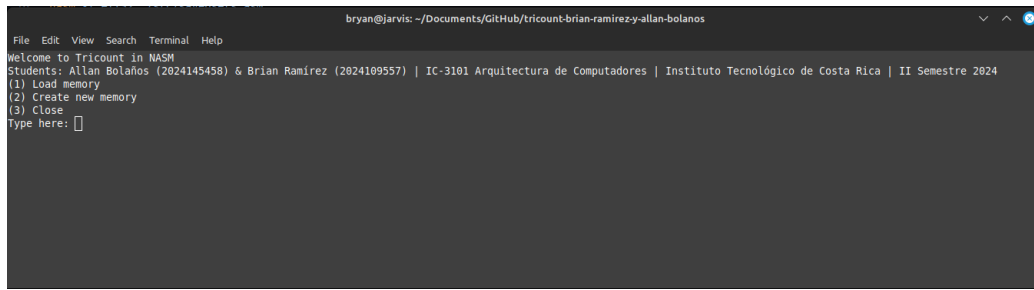


Figure 1: Comando para inicializarse.



```
bryan@jarvis: ~/Documents/GitHub/tricount-brian-ramirez-y-allan-bolanos
File Edit View Search Terminal Help
Welcome to Tricount in NASM
Students: Allan Bolaños (2024145458) & Brian Ramírez (2024109557) | IC-3101 Arquitectura de Computadores | Instituto Tecnológico de Costa Rica | II Semestre 2024
(1) Load memory
(2) Create new memory
(3) Close
Type here: 
```

Figure 2: Pantalla inicial



```
bryan@jarvis: ~/Documents/GitHub/tricount-brian-ramirez-y-allan-bolanos
File Edit View Search Terminal Help
Welcome to Tricount in NASM
Students: Allan Bolaños (2024145458) & Brian Ramírez (2024109557) | IC-3101 Arquitectura de Computadores | Instituto Tecnológico de Costa Rica | II Semestre 2024
(1) Load memory
(2) Create new memory
(3) Close
Type here: 2
Type the name of the new memory: memory
```

Figure 3: Pantalla para solicitud de memoria.

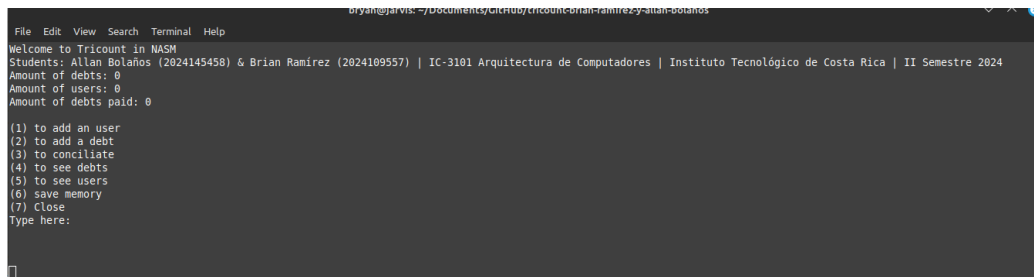


Figure 4: Pantalla para solicitud de memoria.

Dentro de esta pantalla encontrarás distintas opciones:

- (1) To add an user
- (2) To add a debt
- (3) To conciliate
- (4) To see debts
- (5) To see users
- (6) Save memory
- (7) Close

En estas funciones podrás encontrar las opciones básicas del programa donde:

- 1) Podrás añadir un usuario; este te pide el nombre del usuario que quieres añadir y al presionar enter quedará añadido.
- 2) Para añadir una deuda te pedirá que añadas el número de deudores que vas a tener; posteriormente te solicita que establezcas quién es el **fiador**. Y luego según la cantidad de deudores que añadiste irás rellenando con el deudor y su monto. Es importante mencionar que **un fiador puede ser deudor** y **un deudor puede estar más de una vez**, esto debido a que funciona con cambio neto; entonces no alterará el funcionamiento del programa.
- 3) Conciliar: Esta opción te mostrará todos los pagos necesarios para conciliar las deudas de la manera eficiente. Posteriormente eliminará las deudas saldadas de la memoria y podrás continuar añadiendo si así lo deseas.
- 4) En esta opción; podrás mostrar todas las deudas que están cargadas en la memoria.
- 5) En esta opción; podrás ver todos los usuarios cargados en la memoria.
- 6) Guardará las deudas y los usuarios en los respectivos .txt que se crearon al momento de inicializar la memoria.

Conforme vayas añadiendo usuarios, deudas y demás; los valores del header irán actualizándose y mostrándote la información correspondiente.

```
File Edit View Search Terminal Help
Welcome to Tricount in NASM
Students: Allan Bolaños (2024145458) & Brian Ramirez (2024109557) | IC-3101 Arquitectura de Computadores | Instituto Tecnológico de Costa Rica | II Semestre 2024
Amount of debts: 0
Amount of users: 2
Amount of debts paid: 0

(1) to add an user
(2) to add a debt
(3) to conciliate
(4) to see debts
(5) to see users
(6) save memory
(7) close
Type here:
```

Figure 5: Actualización de datos

3 Explicacion de funciones y codigo.

3.1 *Function Request Memory.*

La función request memory utiliza las interrupciones del sistema para solicitar memoria al sistema operativo. La función retorna una dirección de memoria en el registro edx, esa dirección de memoria contiene la primera celda reservada en memoria. La función recibe en la pila como único parámetro por pila, el numero de bytes a reservar en memoria. La función guarda en los primeros cuatro bytes de la estructura, el tamaño total de ese espacio en memoria reservado. Se pretende dividir los espacios reservados en grupo de 4 bytes para usar los registros generales de 4 bytes.



Figure 6: Ejemplo de manejo de memoria con request memory.

3.2 *Function Add First Linked List.*

Esta función recibe por pila la dirección de un puntero a un nuevo nodo recién creado y asume que un puntero a una lista enlazada entra a la función en edx. Lo que hace la función es añadir el nuevo nodo como primero en la lista enlazada o si esta vacía, lo agrega de primero. La función request memory nos permite hacer estructuras relacionadas como memoria dinámica, en nuestro programa utilizamos dos estructuras orientadas a una lista enlazada simple. En la figura 8 y 9 se puede visualizar las estructuras.

3.3 *Conciliate*

En la figura 9; puede observar el código sobre la conciliación. Está primera parte lo que hace es incluir a la pila todos los usuarios que tienen un cambio neto (Previamente

visto en la estructura) negativo. Basandose en la idea de:

Suma de prestado = Suma de recibido. Entonces, desde el primer instante en el que se crea la estructura de usuarios y cómo se maneja; las deudas entre pares (Fiador-Deudor y Deudor-Fiador) están saldadas por defecto. El resto es el principio de pagar hasta que me quede sin dinero lo que genera las cláusulas de la Figura 12. Las cláusulas funcionan restando al monto que se debe el monto debido. Si la resta es positiva; significa que el deudor pagó todo lo que tiene y se retira de la pila. Si la resta es 0; la deuda es saldada y se continua a la siguiente deuda. Si la resta es negativa, significa que el deudor pagó la deuda pero aún tiene dinero por pagar.

De esta forma se concilian todas las deudas de forma eficiente.

3.4 Funcion Create File Folder

La función createFileFolder se utiliza para crear un directorio para cada memoria nueva o sesión, además crea algunos archivos necesarios para guardar información. Utiliza las interrupciones del sistema operativo y recibe varios parámetros por pila como el nombre del directorio y el nombre de los archivos a crear.

En la figura 10 y 11 se puede visualizar el código de la función.

```

1  requestMemory:
2      enter 0,0
3      sub edx,edx
4      mov eax, 45      ;sys_brk
5      sub ebx, ebx
6      int 80h
7
8      mov edx,eax      ;save the address when start of the memory for this structure
9
10
11      add eax, [ebp+8] ;number of bytes to be reserved
12
13      mov ebx, eax      ; save the quantity of bytes in ebx
14      mov eax, 45      ;sys_brk
15      int 80h
16
17      sub eax ,eax
18      mov eax, [ebp+8] ;load the number of bytes to eax
19      mov [edx],eax     ;save the number of bytes of this structure in the first cell of the memory
20
21      cmp eax,0
22      jl  error      ;exit, if error
23
24      mov eax,4
25      sub [edx],eax   ;to have the exact address of the last cell for this structure
26
27      ;By this time, the address of the memory of this structure is in edx
28
29      ;PutStr success_msg
30      jmp exit

```

Figure 7: Comando para inicializarse.



Figure 8: Estructura de deudas.

Tamaño estructura	ID	Nombre	NetChange	Otro Usuario ptr
-------------------	----	--------	-----------	------------------

Figure 9: Estructura de usuarios.

```

1  createFile_Folder:
2
3      enter 0,0
4      ; Create the new directory
5      mov ebx, [ebp+16] ; Load the address of the directory name
6      mov ecx, 0o775    ; Load the permissions into ECX
7      ;7 Read, write, and execute permissions for the owner and group.
8      ;5 Read for others
9
10     sub eax,eax
11     mov eax, 39        ; Syscall number for mkdir -https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md
12     int 80h           ; Call the interrupt,
13
14     ; Check for errors
15     ;A return value of 0 indicates success
16     cmp eax, 0        ; Check if the call was successful
17     jl .error         ; If eax < 0, there was an error
18
19     ;Changing to the new directory
20     sub ebx,ebx
21     mov ebx, [ebp + 16] ; Load the address of the directory name from the stack
22     sub eax,eax
23     mov eax, 12        ; Syscall number for chdir -https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md
24     int 80h           ; Call the interrupt
25
26     ;T000: Could we close the file?, it generates a error
27     ; Create the 3 test files, it could vary
28     mov ebx, [ebp+12] ; Load the address of the file 1 name
29     mov ecx, 0o777
30     sub eax,eax
31     mov eax, 8         ; Syscall number for creat -https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md
32     int 80h
33     ; Close the file
34     mov ebx, eax
35     mov eax, 6         ; syscall: close https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md
36     int 80h           ; Execute the system call

```

Figure 10: Código para crear archivos y folder.

```

1  mov ebx, [ebp+8] ; Load the address of the file 2 name
2  mov ecx, 0x777
3  sub eax, eax
4  mov eax, 8
5  int 80h
6  ; Close the file
7  mov ebx, eax
8  mov eax, 6 ; syscall: close https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md
9  int 80h ; Execute the system call
10
11
12 ;PutStr success_msg
13 ;writeln
14
15 ; Change to the directory
16 mov ebx, returnPath ; Load the address of the directory name
17 mov eax, 12 ; Syscall number for chdir
18 int 0x80 ; Call the interrupt
19
20 ;cleaning
21 sub eax, eax
22 sub ebx, ebx
23 sub ecx, ecx
24 leave
25 ret 16 ; Return to the caller function and clean stack
26
27 .error:
28 ;Print error message
29 PutStr error_msg
30 ;writeln
31
32 ;cleaning
33
34 sub ebx, ebx
35 sub ecx, ecx
36 leave
37 sub eax, eax
38 mov eax, 1 ; Error code
39 ret 16

```

Figure 11: Código para crear archivos y folder.


```

1  addFirstLinkedList:
2      enter 0,0
3      sub ecx,ecx
4      sub eax,eax
5      mov ecx,[ebp+8] ; Load the address of the head of the linked list
6
7      cmp [ecx],eax
8      je .empty
9
10
11
12      ;If the linked list is not empty
13      sub eax,eax
14      mov eax,[edx] ; Load the address of the head of the linked list
15      mov ebx,[ecx] ; Load the address of the new node
16      mov [edx+eax],ebx ; Save the address of the new node in the next pointer of the new node
17      mov [ecx],edx ; Save the address of the new node as the new head of the linked list
18      jmp exit
19
20      .empty:
21          mov [ecx],edx ; Save the address of the new node as the new head of the linked list
22
23
24
25      exit:
26          ;Cleaning
27          sub eax,eax
28          sub ecx,ecx
29          sub ebx,ebx
30          leave
31          ret 4

```

Figure 12: Codigo de la funcion AddFirstLinkedList.

```

1  Conciliate:
2      enter 0,0
3      pusha
4
5      mov eax, [ebp+8] ; Load the pointer to the first node of the linked list STRUCT: [len_struct, ID, name, netChange, ptrNext]
6      mov ebx, [eax]
7      jmp PushNegatives
8
9  PushNegatives:
10     cmp eax, 0
11     je DoSubtractions
12
13     mov ebx, [eax+12] ; Load the net change of the user [STRUCT, ID, name, netChange, ptrNext]
14     cmp ebx, 0 ; Compare the net change of the user with 0
15
16     je NextNode
17     cmp ebx, 0
18     jg NextNode
19
20     push eax
21     jmp NextNode
22
23  NextNode:
24     mov eax, [eax+16] ; Load the next node of the linked list
25     jmp PushNegatives
26
27
28  DoSubtractions:
29     sub ebx, ebx
30     mov edx, [ebp+8]
31     mov edx, [edx]
32
33  MiddleStep:
34     cmp edx, 0
35     je Exit
36
37     mov ecx, [edx+12]
38
39     cmp ecx, 0
40     jl NextPositive
41     cmp ecx, 0
42     je NextPositive
43
44     pop eax ; SEGMENTATION FAULT PROBABLY
45     mov ebx, [eax+12]
46
47     add ecx, ebx
48
49     cmp ecx, 0
50     jg WasGreater
51
52     cmp ecx, 0
53     je WasEqual
54
55     cmp ecx, 0
56     jl WasLess

```

Figure 13: Conciliación parte 1

```

1  WasGreater:
2      PutStr [eax+8]
3      PutStr Payment
4      mov ebx, [eax+12]
5      neg ebx
6      PutLInt ebx
7      PutStr topaying
8      PutStr [edx+8]
9      nwlñ
10     sub ebx, ebx
11     mov [eax+12], ebx
12     mov [edx+12], ecx
13     jmp DoSubstractions
14
15  WasEqual:
16     PutStr [eax+8]
17     PutStr Payment
18     mov ebx, [eax+12]
19     neg ebx
20     PutLInt ebx
21     PutStr topaying
22     PutStr [edx+8]
23     nwlñ
24     sub ebx, ebx
25     mov [eax+12], ebx
26     mov [edx+12], ebx
27     jmp NextPositive
28
29  WasLess:
30     PutStr [eax+8]
31     PutStr Payment
32     PutLInt [edx+12]
33     PutStr topaying
34     PutStr [edx+8]
35     nwlñ
36     sub ebx, ebx
37     mov [eax+12], ecx
38     mov [edx+12], ebx
39     push eax
40     jmp NextPositive
41

```

Figure 14: Conciliación parte 2